Chapter 03 - Cluster Partitioning

7 sections in this chapter:

- Section 3.1: Node Pools
- Section 3.2: Quiz: Node Pools
- Section 3.3: Node Configuration with the Machine Configuration Operator
- Section 3.4: Guided Exercise: Node Configuration with the Machine Configuration Operator
- Section 3.5: Node Configuration with Special Purpose Operators
- Section 3.6: Guided Exercise: Node Configuration with Special Purpose Operators
- Section 3.7: Lab: Cluster Partitioning
- Section 3.8: Summary

Node Pools

Chapter 3 Section 1

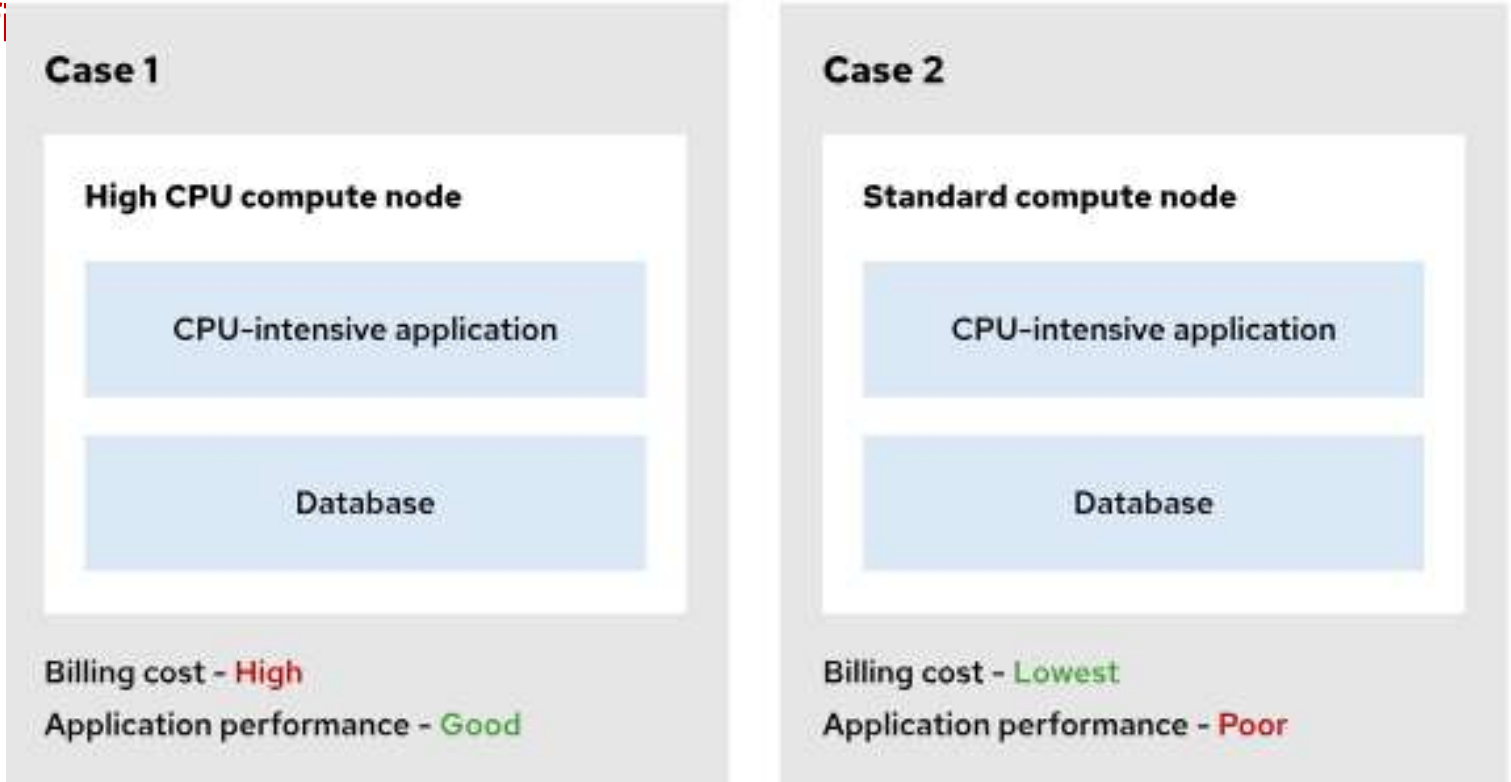
Objectives

After completing this section, you will be:

- Illustrate methods of adding nodes into OpenShift cluster.
- Configuring OpenShift cluster nodes, by using node labels
- Schedule pods to labelled nodes.

Introductions

- Control Plane (Master nodes)
  - Highest hardware resources.
- Data Plane (Worker nodes)
  - Low to high hardware resources.
- Applications (Workloads) have different
  - Specifi

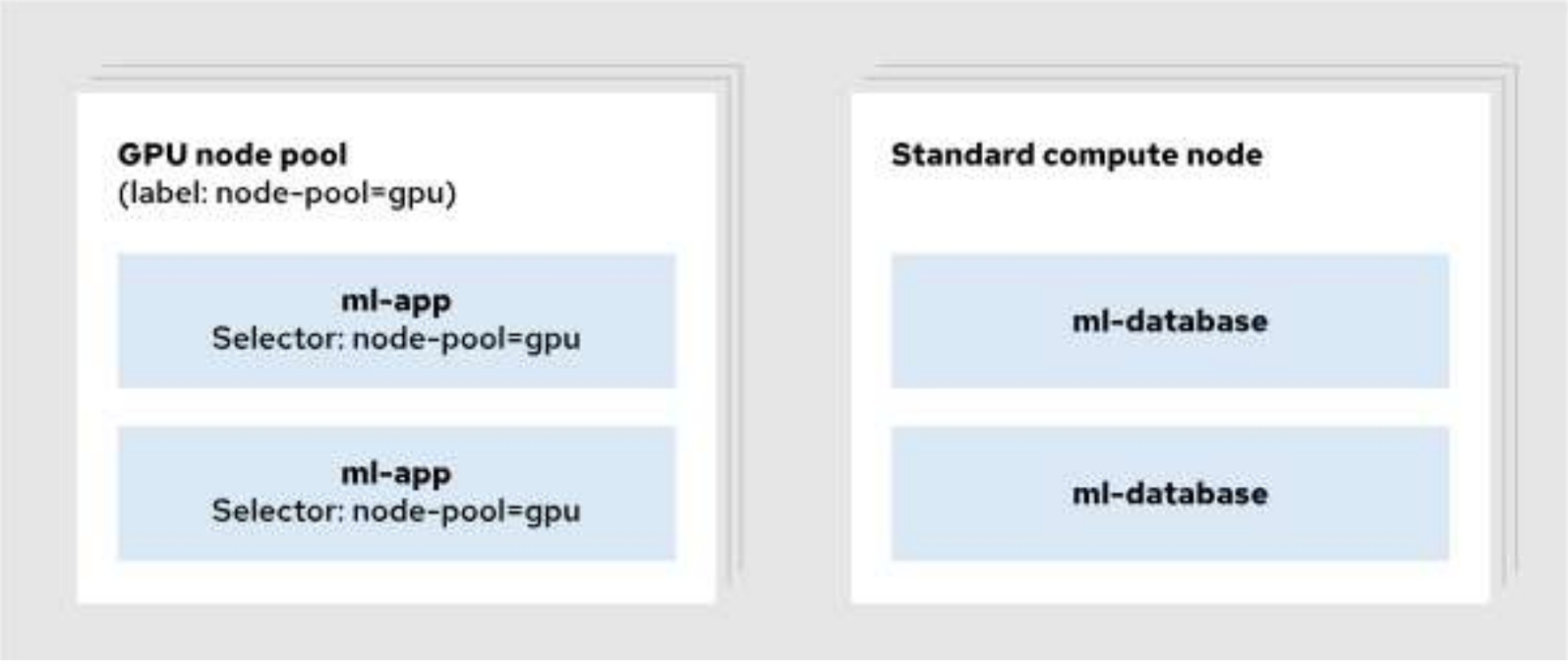


Advantage of grouping applications to nodes

- Allocate resources efficiently for network-intensive applications to mitigate "noisy neighbor" issues.
- Manage costs effectively by labeling expensive hardware.
- Separate infrastructure during maintenance and management.
- Recommended to isolate workloads
  - High performing machine (with GPU)
  - Midrange or Low-budget machine

Node Pools

- Logical group of compute nodes (workers).
- Each pool with similar hardware configuration.
- Easy target for workload placement.
  - Using labels on pods/deployments.



Node Labels (1)

- To assign node to node pool.
- Are key-value pairs attached to node.
- Assign to multiple compute node for one pool.
- Assign label node-pool=gpu to worker01:

```
$ oc label node/worker01 node-pool=gpu
```

```
node/worker01 labeled
```

- List labels of each nodes:

```
$  oc get nodes --show-labels
```

```
NAME        STATUS   ROLES    ...   LABELS
master01    Ready    master          …
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,cluster.ocs.openshift.io/openshift-storage=,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker01,kubernetes.io/os=linux,node-role.kubernetes.io/worker=,node.openshift.io/os_id=rhcos
worker01    Ready    worker   ...
beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,cluster.ocs.openshift.io/openshift-storage=,kubernetes.io/arch=amd64,kubernetes.io/hostname=worker01,kubernetes.io/os=linux,node-role.kubernetes.io/worker=,node.openshift.io/os_id=rhcos,node-pool=gpu
…output omitted…
```

Node Labels (2)

- Assign to multiple compute node for one pool.
- Assign label node-pool=lowcpu to worker02 and worker03:

```
$ oc label nodes worker02 worker03 node-pool=lowcpu
```

- Assign label env=staging to nodes with above labels:

```
$ oc label nodes -l node-pool=lowcpu env=staging
```

- Change value in label env=prod:

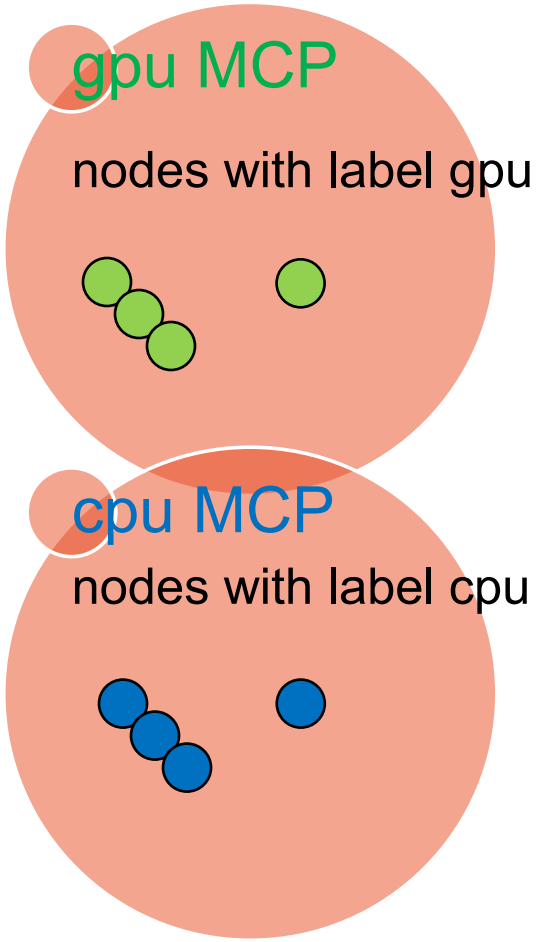
```
$ oc label nodes --selector node-pool=lowcpu env=prod -overwrite
```

- Remove label env from worker03 node:

```
$ oc label nodes worker03 env-
```

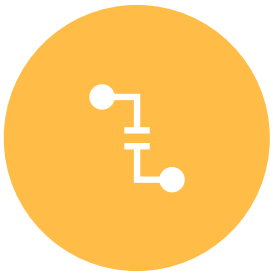
Node Configuration

- OpenShift cluster manage nodes upgrades by:
  - Machine configuration operator (MCO).
- Cluster admin manage nodes upgrades by:
  - Machine configuration pools (MCP).
  - Custom multiple MCPs with labels
- Cloud providers can also provides:
  - Different compute nodes to meet customer requirements
  - Different storage instances to meet customer requirements
  - Details in DO120 (ROSA) and DO121 (Azure)



Section summary

In this section / chapter, you learned:



Segregating pods to different nodes

Node Pools

Node Labelling

Node Provisioning

QUIZ

4 questions

Quiz – 1 of 4

Which is not a default node role for a Red Hat OpenShift Container Platform 4.14 cluster?

A. worker

B. master

C. control-plane

D. storage

Quiz – 1 of 4

Which is not a default node role for a Red Hat OpenShift Container Platform 4.14 cluster?

A. worker

B. master

C. control-plane

D. storage

Quiz – 2 of 4

What is the purpose of using node pools within your OpenShift cluster?

A. Organizing cluster hardware to aid in targeting the most appropriate infrastructure hardware for each component of your deployed applications.

B. Providing an automated monitoring solution for the regions in your data center.

C. Ensuring uptime for workloads by always using the fastest hardware.

D. Alleviating storage constraints for large data sets.

Quiz – 2 of 4

What is the purpose of using node pools within your OpenShift cluster?

A. Organizing cluster hardware to aid in targeting the most appropriate infrastructure hardware for each component of your deployed applications.

B. Providing an automated monitoring solution for the regions in your data center.

C. Ensuring uptime for workloads by always using the fastest hardware.

D. Alleviating storage constraints for large data sets.

Quiz – 3 of 4

Which command adds a compute node to a node pool?

A. oc set label node/NODE_NAME pool=NODE_POOL_NAME

B. oc label node/NODE_NAME node-pool=NODE_POOL_NAME

C. oc get node NODE_NAME --selector node-pool=NODE_POOL_NAME

D. oc create node-pool=NODE_POOL_NAME NODE_NAME

Quiz – 3 of 4

Which command adds a compute node to a node pool?

A. oc set label node/NODE_NAME pool=NODE_POOL_NAME

B. oc label node/NODE_NAME node-pool=NODE_POOL_NAME

C. oc get node NODE_NAME --selector node-pool=NODE_POOL_NAME

D. oc create node-pool=NODE_POOL_NAME NODE_NAME

Quiz – 4 of 4

Which is not an advantage of using node pools when running applications?

A. Allocate resources efficiently for network-intensive applications to mitigate "noisy neighbor" issues.

B. Automate security of applications in the cluster.

C. Manage costs effectively by labeling expensive hardware.

D. Separate infrastructure during maintenance and management.

Quiz – 4 of 4

Which is not an advantage of using node pools when running applications?

A. Allocate resources efficiently for network-intensive applications to mitigate "noisy neighbor" issues.

B. Automate security of applications in the cluster.

C. Manage costs effectively by labeling expensive hardware.

D. Separate infrastructure during maintenance and management.

Node Configuration with the Machine Configuration Operator

Chapter 3 Section 3

Objectives

After completing this section, you will be:

- Apply operating system settings to cluster nodes
  - with the machine configuration operator (MCO).

The Machine Configuration Operator (MCO) (1)

- Configure Red Hat Enterprise Linux CoreOS (RHCOS) in nodes.
  - Update entire OS as image (instead of package-by-package basis).
  - Manage OS configuration changes.
- Do not perform this onto nodes:
  - Manually edit files.
  - Run systemctl command.
  - Run dnf/yum/rpm command.
- MCO comprises following pods:
  - Machine-config-operator (MCO)

    Main operator that manages rest of MCO components.
  - Machine-config-controller (MCC)

    Synchronizes machines upgrades
  - Machine-config-server (MCS)

    Provides instance customization to machines that newly joined node.

The Machine Configuration Operator (MCO) (2)

- Can manage following resources:
  - Systemd services
  - SSH keys
  - Kernel arguments
  - Files in /var and /etc directories
  - Files in /opt and /usr/local directories

- Has two custom resources:
  - A machine configuration (MC) CR

    Declares instance customizations.
  - Machine configuration pool (MCP) CR

    Uses label to match one or more MCs to group of nodes.

    Create pool of nodes using machineConfigSelector and nodeSelector parameters.

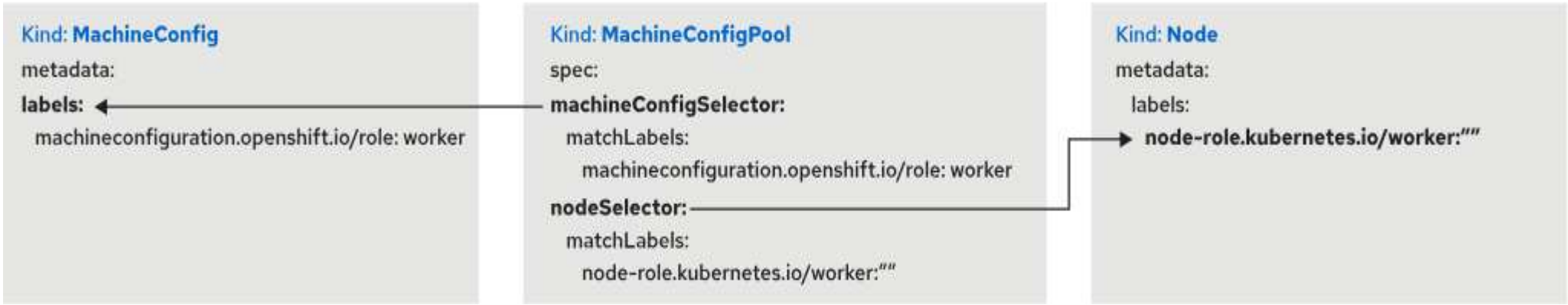
Relationship between node, MC and MCP labels

- Has two custom resources:
  - A machine configuration (MC) CR

    Declares instance customizations.
  - Machine configuration pool (MCP) CR

    Uses label to match one or more MCs to group of nodes.

    Create pool of nodes using machineConfigSelector and nodeSelector parameters.



Machine Configurations (1)

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 60-journald
spec:
  config:
    ignition:
...continue...>
spec:
...output omitted...
    storage:
      files:
      - contents:
          source: data:text/plain;charset=utf-8;base64,VGVzdGl...E8gKltAo=
        filesystem: root
        mode: 0644
        path: /etc/systemd/journald.conf
```

Machine Configurations (2)

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: 99-kernel-realtime
spec:
  kernelType: realtime
…output omitted…
```

Get more details on MC

- List all MCs:

  $ oc get machineconfig or oc get mc command.

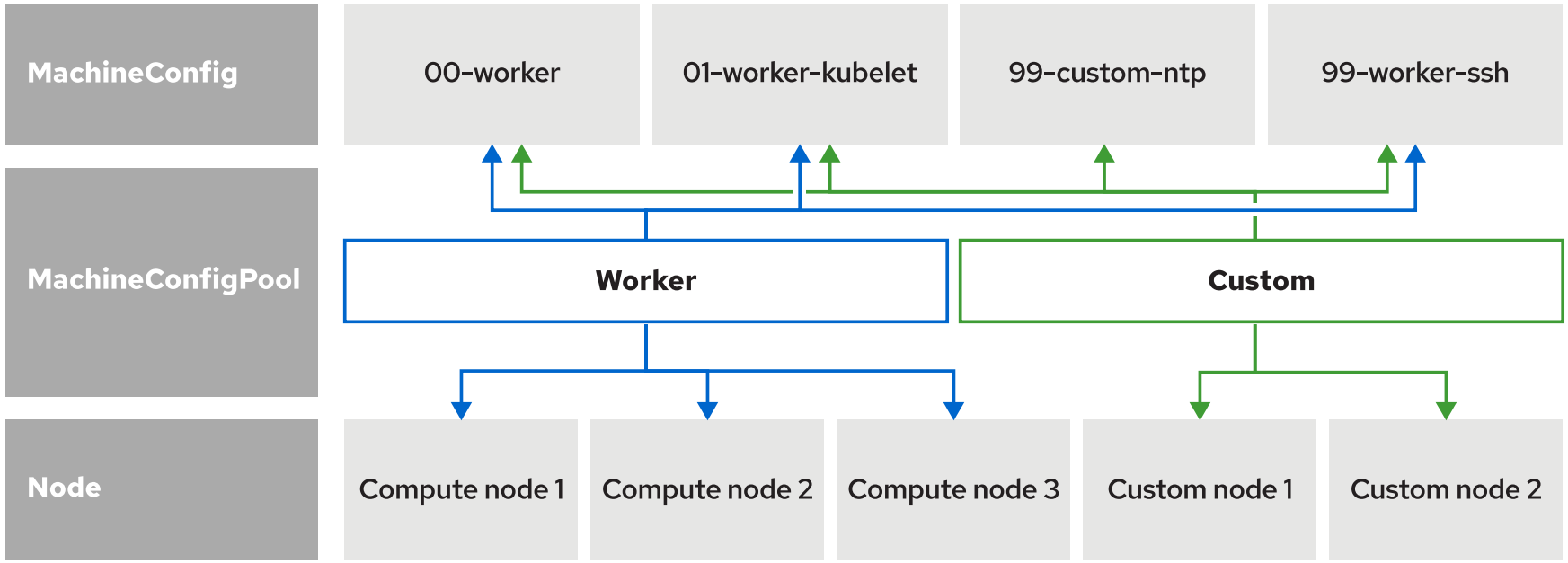
- List all MCs specific to node label:

  $ oc get mc --selector machineconfiguration.openshift.io/role=worker

NAME	GENERATEDBYCONTROLLER	IGNITIONVERSION	AGE
00-worker	52fe...bf97	3.2.0	108d
01-worker-container-runtime	52fe...bf97	3.2.0	108d
01-worker-kubelet	52fe...bf97	3.2.0	108d
98-worker-generated-registries	52fe...bf97	3.2.0	108d
99-worker-chrony-conf-override	..	3.2.0	108d
99-worker-ssh	…	3.2.0	108d

Machine Configuration Pools (MCP)

- Use labels to match one or more MCs to one or more nodes.
- Split node's configuration.
- Focus every MC on one aspect of server configuration:
  - One MC for DNS resolution.
  - One MC for time synchronization.



Creating separate pool for custom nodes

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
   name: custom <1>
spec:
   machineConfigSelector: <2>
      matchExpressions:
         - key: machineconfiguration.openshift.io/role
           operator: In
           values: [worker, custom] <2>
   nodeSelector: <3>
      matchLabels:
         node-role.kubernetes.io/custom: "“ <3>
```

<1> Name for the MCP is custom

<2> MC selector includes both worker and custom MCs

<3> Node selector includes nodes in cluster with custom role

Get more details on MCP (1)

- List all MCPs:

  $ oc get machineconfigpool or oc get mcp command.

- List all MCs specific to node label:

  $ oc get mcp --selector machineconfigurationpool.openshift.io/role=worker

```
NAME    CONFIG                 UPDATED  UPDATING  DEGRADED
MACHINECOUNT  READYMACHINECOUNT  UPDATEDMACHINECOUNT
    DEGRADEDMACHINECOUNT  AGE
master  rendered-master-716...267  True     False     False
3             3                 3                   108d
worker  rendered-worker-573...3db  True     False     False
3             3                 3                   108d
```

Get more details on MCP (2)

- List all MCPs:

  $ oc get machineconfigpool or oc get mcp command.

- List all MCs specific to node label:

  $ oc get mcp --selector machineconfigurationpool.openshift.io/role=worker

```
NAME    CONFIG                 UPDATED  UPDATING  DEGRADED
MACHIN
    DEG
master  r
3
worker  r
3
```

CONFIG

- Name for the rendered MC

UPDATED=true

- MCO has been applied current MC to nodes successfully

UPDATED=false

- MCO is still updating / applying MC to nodes.

DEGRADED:

- **True** : MCO blocked from applying configuration.
- **False**: all is good and ready

Get more details on MCP (2)

- List all MCPs:

  $ oc get machineconfigpool or oc get mcp command.

- List all MCs specific to node label:

  $ oc get mcp --selector machineconfigurationpool.openshift.io/role=worker

```
NAME    CONFIG        …          … MACHINECOUNT  READYMACHINECOUNT
    UPDATEDMACHINECOUNT  DEGRADEDMACHINECOUNT  AGE
master  rendered-master-716…267  True…  False…  False
3
worker  r
3
```

MACHINECOUNT

- Total number of machines in MCP.

READYMACHINECOUNT

- Total number of machines in MCP ready for scheduling.

UPDATEDMACHINECOUNT

- Total number of machines updated with current MC.

DEGRADEDMACHINECOUNT

- Total number of machines degraded or blocked or irreconcilable.

AGE

- Last status updated

Label Nodes role

- Add custom role to worker03:

  $ oc label node/worker03 node-role.Kubernetes.io/custom=

- Verify node roles:

  $ oc get nodes

NAME	STATUS	ROLES	AGE	VERSION
master01	Ready	control-plane,master	114d	v1.25.7+eab9cc9
master02	Ready	control-plane,master	114d	v1.25.7+eab9cc9
master03	Ready	control-plane,master	114d	v1.25.7+eab9cc9
worker01	Ready	worker	12d	v1.25.7+eab9cc9
worker02	Ready	worker	12d	v1.25.7+eab9cc9
worker03	**Ready**	**custom,worker**	**12d**	**v1.25.7+eab9cc9**

Infrastructure Nodes

- Specialized nodes to run infrastructure-related functions:
  - Monitoring & Logging.
  - Providing OAuth services or storage.
  - Facilitate network communication within cluster.
  - Other critical cluster services.
- Label using infra role.
- Optional, but recommended for large clusters:
  - Ensure performance and stability.
- Infra node requires:
  - No OpenShift subscriptions.
  - Custom MCP to apply separate configuration.

Machine Configuration Operator Updates

- Handle by machine configuration daemons (MCD):
  - Update and validate configuration to each node
  - Reports state of node updates by using annotations.
- Cluster-admin query this annotations to assess state of update.

$ oc describe node worker01

```
Name:         worker01
Roles:        worker
Labels:       beta.kubernetes.io/arch=amd64
              ...output omitted...
              node-role.kubernetes.io/worker=
              node.openshift.io/os_id=rhcos
Annotations:  machineconfiguration.openshift.io/currentConfig: rendered-worker-370...bfd
              machineconfiguration.openshift.io/desiredConfig: rendered-worker-370...bfd
              machineconfiguration.openshift.io/reason:
              machineconfiguration.openshift.io/state: Done
              volumes.kubernetes.io/controller-managed-attach-detach: true
...output omitted...
```

Configuration Drift

- Current MC not matching Desired MC.
- MCD drain, update, reboot and check state of nodes.
- Check for following upon reboot of nodes:
  - Any files in MC modified outside of MC.
  - Current MC not maching Desired MC.
- If configuration drifts deteced, MCD performs:
  - Write to logs.
  - Generate a Kubernetes events.
  - Marks nodes and MCP with degraded state
  - Stop additional drift detection on affected nodes.
- Degraded nodes:
  - Cannot update.
  - Still online and operational.

Remediate degraded nodes

Generate force file (**/run/machine-config-daemon-force**):

- ✓ Force degraded node to bypass configuration drift detection.
- ✓ Re-apply current MC.

Rewrite

- ✓ Change file permissions or modify files to match MC configuration manually.

```
...output omitted...
Status:
  Conditions:
    ...output omitted...
    Last Transition Time:  2023-10-02T10:11:37Z
    Message:               Node worker01 is reporting: "content mismatch for file
      '/etc/containers/registries.conf'"
    Reason:                1 nodes are reporting degraded status on sync
    Status:                True
    Type:                  NodeDegraded
    Last Transition Time:  2023-10-02T10:11:37Z
    Message:
    Reason:
```

Example of configuration drift:

```
...output omitted...
Status:
  Conditions:
    ...output omitted...
    Last Transition Time:  2023-10-02T10:11:37Z
    Message:               Node worker01 is reporting: "content mismatch for file
      '/etc/containers/registries.conf'"
    Reason:                1 nodes are reporting degraded status on sync
    Status:                True
    Type:                  NodeDegraded
    Last Transition Time:  2023-10-02T10:11:37Z
    Message:
    Reason:
    Status:                True
    Type:                  Degraded
...output omitted...
```

Remediation step (1)

- List pods affected by node:

```
$ oc get pod -n openshift-machine-config-operator \
 --field-selector spec.nodeName=worker01
```

```
NAME                          READY   STATUS    RESTARTS   AGE
machine-config-daemon-jsrzm   2/2     Running   2          19d
```

- Review logs in the pod:

```
$ oc logs machine-config-daemon-jsrzm  -n openshift-machine-config-operator
```

```
...output omitted...
I1002 10:11:40.510208    2667 rpm-ostree.go:394] Running captured: rpm-ostree kargs
E1002 10:11:40.568388    2667 on_disk_validation.go:207] content mismatch for file
    "/etc/containers/registries.conf" (-want +got):  []uint8(
    """
-   unqualified-search-registries = ["registry.access.redhat.com", "docker.io"]
+   unqualified-search-registries = ["docker.io"]
    short-name-mode = ""
...output omitted...
E1002 10:11:46.987190    2667 daemon.go:1077] mode mismatch for file:
    "/etc/containers/registries.conf"; expected: -rw-r--r--/420/0644; received: -rwxrwxrwx/511/0777
```

Configure other MCO-related custom resources

- The ContainerRuntimeConfig CR:
  - CRI-O container runtime settings.
- The KubeletConfig CR:
  - Kubelet service settings.

- Settings examples:
  - Container runtime

    Set container runtime to either runc (default) or crun.
  - Loglevel

    Sets level of verbosity for logging messages.
  - Overlay size

    Sets maximum size of container image.
  - PID limits

    Sets maximum of processes.

Section summary

In this section / chapter, you learned:

About the Machine Configuration Operator (MCO)

About Machine configuration Pools (MCP)

About Machine Configurations (MC)

About Configuration Drift

About Machine Configuration Daemon (MCD)

Configure other MCO-related CR

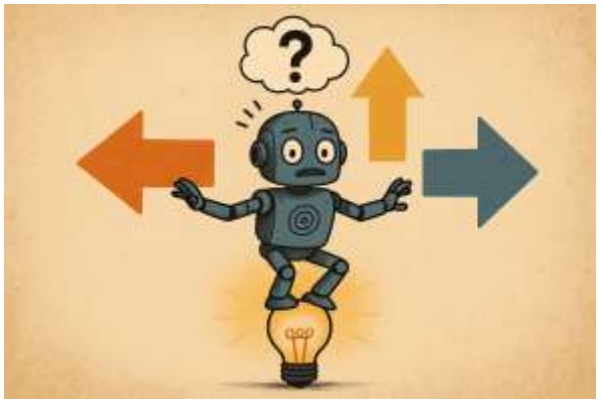
Section 3.4 Guided Exercise Node Configuration with the Machine Configuration Operator

You should be able to:

- Use the machine configuration operator (MCO) to modify the Network Time Protocol (NTP) client configuration.
- Create a machine configuration pool (MCP) and a machine configuration (MC).
- Observe the status of MC updates.

Estimated completion time: 10-15 minutes

Continue / Guided Exercises / Break?



Node Configuration with Special Purpose Operators

Chapter 3 Section 5

Objectives

After completing this section, you will be:

- Apply operating system settings to cluster nodes
  - with higher-level operators instead of the low-level machine configuration operator.

Special Purpose Operators

- Focus higher level configuration
  - Cannot be handled thru MCO.
  - Specialized hardware tuning.
  - Cluster organization.
  - Advanced optimization.
- Runs in designated namespace
- Implement using custom resource definitions (CRD) and custom resource (CR).
- Can be integrated with service account to grant permissions across namespace.
- Example of Special Purpose Operators:
  - ❑ Node Tuning Operator
  - ❑ Node Feature Discovery Operator
  - ❑ Kernel Module Management Operator
  - ❑ Vendor-provided Operators

The Node Tuning Operator

- A cluster-version operator. Installed by default.
- Manages node-level optimization process.
- Uses the TuneD service and tuned daemon.
- Manages containerized tuned daemon configured as daemon set.
- Custom TuneD profiles:
  - Granular customization of nodes for different environments.
- Verify it is installed and operational:

  $ oc get tuned/default –o yaml –n openshift-cluster-tuning-operator
- Vied applied profiles for each node:

  $ oc get profile -n openshift-cluster-tuning-operator

TuneD profile (1)

- Optimize node accordingly to parameters & plug-in.

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: openshift-network-tuning
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - name: openshift-network-tuning
      data: |
        [main]
        summary=Optimize network parameters for OpenShift worker nodes
        include=openshift-node
        [sysctl]
        net.core.rmem_default = 524287
        net.core.wmem_default = 524287
        net.core.optmem_max = 524287
        net.core.netdev_max_backlog = 300000
  recommend:
…continue…>
```

TuneD profile (2)

- Optimize node accordingly to parameters & plug-in.

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: openshift-network-tuning
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - name: openshift-network-tuning
      data: |
        [main]
        summary=Optin
        include=opensh
        [sysctl]
        net.core.rmem_
        net.core.wmem_default = 524287
        net.core.optmem_max = 524287
        net.core.netdev_max_backlog = 300000
  recommend:
…continue…>
```

```
…continue…>
 recommend:
    - match:
        - label: node-role.kubernetes.io/worker
      priority: 20
      profile: openshift-network-tuning
```

The Node Feature Discovery Operator (NFD)

- Runs in the openshift-nfd namespace
- Expose node-level information.
- Detects hardware details:
  - Processor architecture
  - Specialized PCI hardware (GPU)
  - Networking interface cards (SR-IOV, MR-IOV, vFC, TCP engine off-loading)
- Gather information about detected hardware:
  - PCI make, model, vendor
  - CPU specifications
  - Kernel information
  - Operating system version and etc.
- Add labels to nodes with gathered information:
  - To aid deployment placement.
  - To be handled specially by MCO.

The Kernel Module Management Operator (KMM)

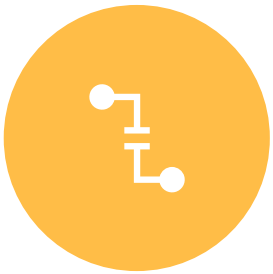
- Can be installed and runs in the openshift-kmm namespace.
- Manage, builds, signs and deploys out-of-tree kernel modules or device plug-ins.
- Adding extra, alternative or custom modules to cluster nodes.
- Case use:
  - New technology introduced.
  - Custom kernel modules for special computational optimization operations.

Vendor-provided Operators

- Hardware vendor author special purpose operators for openshift.
- Provide unique functions that correspond to their hardware.
- Cases:
  - IBM, Broadcom, Nvidia, Intel, ARM and etc.

Section summary

In this sectio, you have learned:



Segregating pods to different nodes

Node Pools

Node Labelling

Node Provisioning

END OF CHAPTER

Section 3.6 & 3.7

Objectives:

- Section 3.6: Guided Exercise: Node Configuration with Special Purpose Operators
- Section 3.7: Lab: Cluster Partitioning

Estimated completion time: 45-60 minutes

Continue / Break?